

doc

Name: Finnldy

Entwickler: Muhammet Altuntas / Jakob Seeberger

Projektplan

Projektbeschreibung:

Die Anwendung ist eine Gruppen-Filmempfehlungs-App wie „Tinder für Filme“. Nutzer können gemeinsam mit Freunden oder Familie einer Lobby beitreten. Anschließend werden allen Teilnehmern Filme aus einer Datenbank vorgeschlagen. Jeder Nutzer kann einen Film liken, disliken oder als „schon gesehen“ markieren. Sobald alle Teilnehmer denselben Film geliked haben, wird dieser als gemeinsames Ergebnis angezeigt. Filme, die von einem Nutzer als „schon gesehen“ markiert wurden, werden aus der Auswahl ausgeschlossen. Falls nach 50 Swipes kein perfekter Treffer gefunden wurde, wird der Film mit der besten Bewertung angezeigt. Zusätzlich gibt es „Honorable Mentions“, bei denen die Plätze 2 bis 5 angezeigt werden. Nutzer können Filme außerdem zu einer „Später ansehen“-Liste hinzufügen und im Hauptmenü ihre gesehenen sowie gespeicherten Filme ansehen.

Zuständigkeiten

Jakob:

BII:

- Lobby
- Swipe-Klassen
- resultklassen

GUI:

- Startseite(Fenster) & kleines Lobby fenster

DAL:

- Database-Klasse(alles was mit laden/Lesen zutun hat)
- Ireposotory
- Networkservive client-teil

Muhammet:

BII:

- Users-Klasse
- Movie + Rep

GUI:

- Filmkarte anzeigen(Cover, Titel, Beschreibung, Genres)
- Nebenfenster(Schongesehen, resultview usw.)

DAL:

- Networkservice Host-teil
- Databse-Klasse(alles was mit speichern zu tun hat)

Gemeinsam:

- Lobby-Klasse
- LobbyController-Klasse
- Temporäre Sachen wie swipe view oder Resultviewe

Was wer tatsächlich gemacht hat

Jakob:

- Die ganze swipe logik mit Filter & Anbindung an UI
- Ganze UI
- Ergebniss Logik
- DBI und POS Anbindung
- Simples Interface

Muhammet:

- Daten übertragung
- Verbindung zwischen Laptops
- Controller
- Tests
- Methoden für die anbindung

Milestones

bis 24.05:

- Movie +Repository
- users
- Startseite
- LobbyController
- Startfenster
- Filmkarten-GUI Grundlayout

anfangen:

- Database Lesen/Schreiben
- swipeview
- Resultview

bis 31.05

- Movies laden
- API-Verbindung
- JSON lesen/deserialisieren
- resultKlassen
- swipeklassen

anfangen:

- Lobby
- HandyLobby
- Networking

fertig machen von:

- Database Lesen/Schreiben

bis 07.06

- HandyGUI

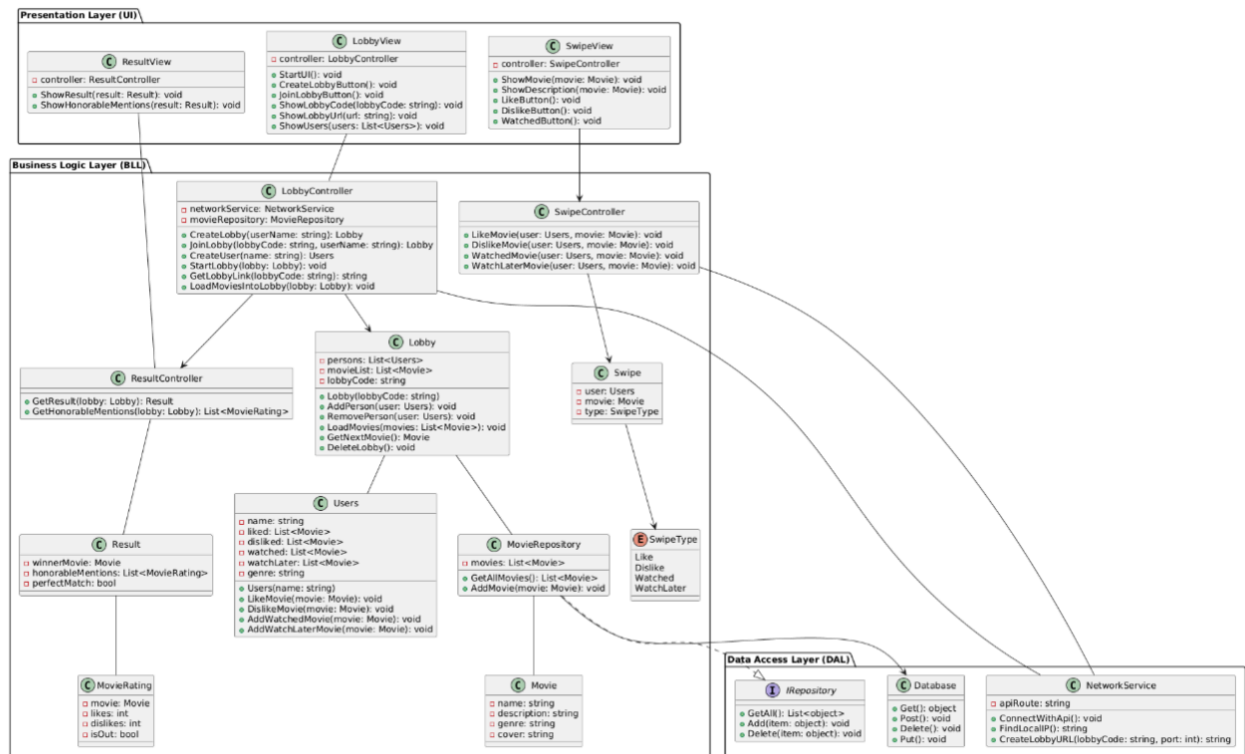
fertig machen von:

- Lobby
- HandyLobby

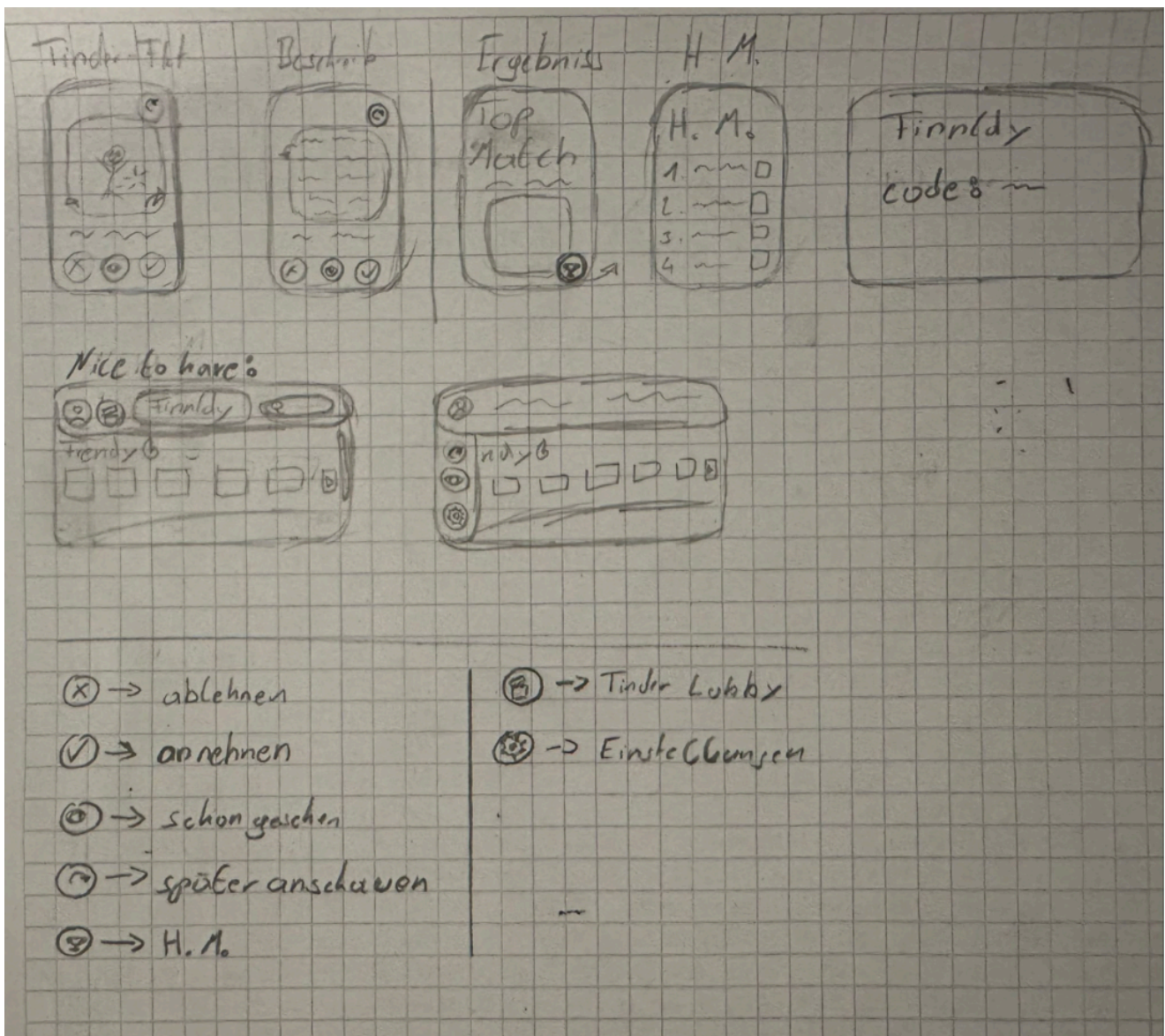
Fazit nach dem Projekt ob mir die Milestones eingehalten haben

Wie haben die Zeit unterschätzt die mir für die Logik und alles ohne Ki brauchen darum waren wir hinter der Milestone Zeit. Das mit dem Handy haben wir nicht geschafft dafür fehlte uns einfach die Zeit und das Wissen.

UMLdiagramm



UI



Was haben wir uns dabei gedacht?

Wir haben unser UML in drei Schichten unterteilt weil

- UI --> Dort liegen alle Fenster wie LobbyView oder SwipeView. Diese Klassen sollen für das ganze Aussehen und UI Logik zuständig sein.
- BLL --> Dort sind Klassen wie LobbyController oder ResultController. Hier passiert die meiste Logik vom Code z.B User zur Lobby hinzufügen oder Filme aus der API herunterladen.
- DAL --> Die Idee war alles was mit Speichern, Laden oder untereinander Verbinden zu tun hat in DAL zu machen und es von BLL ab zu kapseln.

Über was haben wir viel Diskutiert?

- Über die Verbindungen zwischen den einzelnen Klassen
- Die Controller waren am Anfang noch nicht detailliert genug
- Ob mir Host und Client NetworkService in eine Klasse packen oder nicht. haben es im Uml in eine Klasse gepackt aber beim Coden gemerkt er wäre sinnvoller gewesen einzelne Klassen zu machen weil sie nichts miteinander zu tun haben.

Was war gut am UML?

- Die wichtigsten Sachen im Code wurden auch so im UML dargestellt z.B Lobby erstellen, User erstellen , Ergebnis berechnen, Filme Swipen
- Das wir einige Controller Klassen geplant haben
- Das wir SwipeType als Enum genommen haben

Was war schlecht gemacht im UML?

- Network war im UML viel zu vereinfacht dargestellt auch nicht durchdacht. Wir unterscheiden jetzt zwischen Host und Client
- Wir haben kein GenreSelectWindow gemacht und das ist ein großer und wichtiger Bestandteil für den Filter und das Swipen.
- Ein paar Klassen wie z.B IRepository wurden zwar geplant aber nie umgesetzt dadurch das geplant war das dort die Daten gespeichert gelöscht und geladen werden wir das aber logischerweise in FastAPI in DBI gemacht haben.